

Introduzione alla Crittografia

Crittografia Simmetrica

La crittografia simmetrica, definita anche crittografia convenzionale o a chiave singola, era l'unico tipo di crittografia in uso prima dell'introduzione della crittografia a chiave pubblica alla fine degli anni '70. Innumerevoli individui e gruppi, da Giulio Cesare alla forza degli U-boat tedeschi fino agli odierni utenti diplomatici, militari e commerciali, hanno utilizzato la crittografia simmetrica per comunicazioni segrete. Essa rimane il tipo di crittografia più ampiamente utilizzato tra i due.

Uno schema di crittografia simmetrica ha cinque componenti:

- **Testo in chiaro:** Questo è il messaggio originale o i dati che vengono inseriti nell'algoritmo come input.
- **Algoritmo di cifratura:** L'algoritmo di cifratura esegue varie sostituzioni e trasformazioni sul testo in chiaro.
- **Chiave segreta:** Anche la chiave segreta è un input per l'algoritmo di cifratura. Le esatte sostituzioni e trasformazioni eseguite dall'algoritmo dipendono dalla chiave.

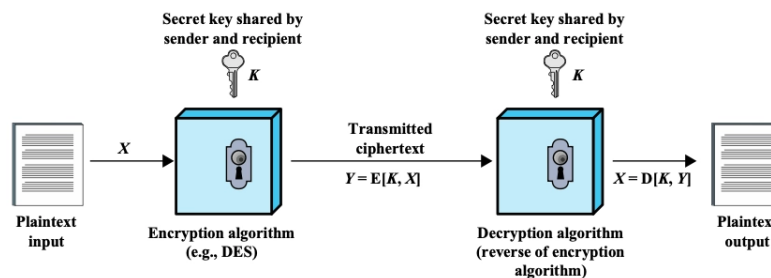


Figure 2.1 Simplified Model of Symmetric Encryption

- **Algoritmo di decifratura:** Si tratta essenzialmente dell'algoritmo di cifratura eseguito al contrario. Prende il testo cifrato e la chiave segreta e produce il testo in chiaro originale.
- **Testo cifrato:** Questo è il messaggio rimescolato prodotto come output. Dipende dal testo in chiaro e dalla chiave segreta. Per un dato messaggio, due chiavi diverse produrranno due testi cifrati diversi.

Esistono due requisiti per l'uso sicuro della crittografia simmetrica:

1. Abbiamo bisogno di un forte algoritmo di cifratura. Come minimo, vorremmo che l'algoritmo fosse tale che un avversario che conosce l'algoritmo e ha accesso a uno o più testi cifrati non sia in grado di decifrare il testo cifrato o di risalire alla chiave.

2. Il mittente e il destinatario devono aver ottenuto copie della chiave segreta in modo sicuro e devono mantenere la chiave segreta. Se qualcuno riesce a scoprire la chiave e conosce l'algoritmo, tutte le comunicazioni che utilizzano questa chiave sono leggibili.

Esistono due approcci generali per attaccare uno schema di crittografia simmetrica. Il primo attacco è noto come **criptoanalisi**. Questo tipo di attacco sfrutta le caratteristiche dell'algoritmo per tentare di dedurre uno specifico testo in chiaro o per dedurre la chiave utilizzata. Se l'attacco riesce a dedurre la chiave, l'effetto è catastrofico: tutti i messaggi futuri e passati crittografati con quella chiave sono compromessi.

Il secondo metodo, noto come **attacco a forza bruta (brute-force attack)**, consiste nel provare ogni possibile chiave su un pezzo di testo cifrato finché non si ottiene una traduzione intelligibile in testo in chiaro. In media, devono essere provate metà di tutte le chiavi possibili per ottenere il successo.

Algoritmi di Cifratura Simmetrica a Blocchi

Gli algoritmi di crittografia simmetrica più comunemente usati sono i cifrari a blocchi. Un cifrario a blocchi elabora l'input in chiaro in blocchi di dimensione fissa e produce un blocco di testo cifrato di uguale dimensione per ogni blocco di testo in chiaro. L'algoritmo elabora quantità di testo in chiaro più lunghe come una serie di blocchi di dimensione fissa. I più importanti algoritmi simmetrici, che sono tutti cifrari a blocchi, sono il Data Encryption Standard (DES), il Triple DES e l'Advanced Encryption Standard (AES).

	DES	Triple DES	AES
Dimensione blocco testo in chiaro (bit)	64	64	128
Dimensione blocco testo cifrato (bit)	64	64	128
Dimensione chiave (bit)	56	112 o 168	128, 192 o 256

I. DATA ENCRYPTION STANDARD

Il **Data Encryption Standard (DES)**, adottato dal NIST nel 1977 e basato sull'algoritmo **DEA**, è stato per decenni lo schema di crittografia simmetrica di riferimento, operando su blocchi di testo in chiaro di **64 bit** con una chiave di **56 bit**. Nonostante sia l'algoritmo più studiato della storia, la sua struttura logica non ha mai mostrato debolezze fatali dal punto di vista della crittoanalisi teorica; tuttavia, la sua criticità principale risiede esclusivamente nella **ridotta lunghezza della chiave**. Con sole 2^{56} combinazioni possibili (circa $7,2 \times 10^{16}$), il DES è oggi estremamente vulnerabile agli **attacchi a forza bruta**: se un singolo PC moderno può forzarlo in circa un anno, la potenza di calcolo dei supercomputer attuali (capaci di testare 10^{13} chiavi al secondo) può individuare la chiave in **circa un'ora**. Al contrario, standard moderni come l'**AES** utilizzano chiavi da almeno **128 bit**, rendendo vano qualsiasi attacco a forza bruta anche ipotizzando un incremento della velocità di calcolo di un trilione di volte. Per ovviare a queste obsolescenze, la crittografia si è evoluta verso sistemi più robusti come il **Triple DES (3-DES)** e, definitivamente, verso l'**AES (Advanced Encryption Standard)**.

II. TRIPLE DES

La vita del DES è stata estesa dall'uso del Triple DES (3DES), che prevede la ripetizione dell'algoritmo DES di base tre volte, utilizzando due o tre chiavi uniche, per una dimensione della chiave di 112 o 168 bit. Il 3DES ha due attrattive che ne spiegano la diffusione negli anni successivi. In primo luogo, con la sua lunghezza di chiave di 168 bit, supera la vulnerabilità all'attacco a forza bruta del DES. In secondo luogo, l'algoritmo di cifratura sottostante in 3DES è lo stesso del DES. Questo algoritmo è stato sottoposto a più controlli rispetto a qualsiasi altro algoritmo di cifratura su un periodo di tempo più lungo, e non è stato trovato alcun attacco crittoanalitico efficace basato sull'algoritmo piuttosto che sulla forza bruta.

Di conseguenza, vi è un alto livello di fiducia che il 3DES sia molto resistente alla crittoanalisi. Se la sicurezza fosse l'unica considerazione, allora il 3DES sarebbe una scelta appropriata per un algoritmo di cifratura standardizzato per i decenni a venire. Il principale svantaggio del 3DES è che l'algoritmo è relativamente lento nel software. Il DES originale è stato progettato per implementazioni hardware della metà degli anni '70 e non produce un codice software efficiente. Il 3DES, che richiede tre volte più calcoli del DES, è corrispondentemente più lento. Un secondo svantaggio è che sia il DES che il 3DES utilizzano una dimensione del blocco di 64 bit. Per ragioni sia di efficienza che di sicurezza, è auspicabile una dimensione del blocco maggiore.

III. ADVANCED ENCRYPTION STANDARD

A causa dei suoi svantaggi, il 3DES non è un candidato ragionevole per l'uso a lungo termine. In sostituzione, ci sono state richieste di proposte per un nuovo Advanced Encryption Standard (AES), che dovrebbe avere una forza di sicurezza uguale o migliore del 3DES e un'efficienza significativamente migliorata. Oltre a questi requisiti generali, il NIST ha specificato che l'AES deve essere un cifrario a blocchi simmetrico con una lunghezza del blocco di 128 bit e supporto per lunghezze di chiave di 128, 192 e 256 bit. I criteri di valutazione includevano sicurezza, efficienza computazionale, requisiti di memoria, idoneità hardware e software e flessibilità. In una prima fase di valutazione, sono stati accettati 15 algoritmi proposti. Un secondo round ha ristretto il campo a 5 algoritmi. L'AES è ora ampiamente disponibile nei prodotti commerciali.

Dimensione chiave (bit)	Numero di chiavi alternative	Tempo richiesto a 10^9 decifrazioni/s	Tempo richiesto a 10^{13} decifrazioni/s
56 (DES)	$2^{56} = 7.2 \times 10^{16}$	2^{55} ns = 1.125 anni	1 ora
128 (AES)	$2^{128} = 3.4 \times 10^{38}$	2^{127} ns = 5.3×10^{21} anni	5.3×10^{17} anni
168 (Triple DES)	$2^{168} = 3.7 \times 10^{50}$	2^{167} ns = 5.8×10^{33} anni	5.8×10^{29} anni
192 (AES)	$2^{192} = 6.3 \times 10^{57}$	2^{191} ns = 9.3×10^{40} anni	9.3×10^{36} anni
256 (AES)	$2^{256} = 1.2 \times 10^{77}$	2^{255} ns = 1.8×10^{59} anni	1.8×10^{55} anni

Cifrari a Flusso

Un **cifrario a blocchi** elabora l'input un blocco di elementi alla volta, producendo un blocco di output per ogni blocco di input. Un **cifrario a flusso** elabora gli elementi di input continuamente, producendo un elemento di output alla volta, man mano che procede. Sebbene i cifrari a blocchi siano molto più comuni, esistono alcune applicazioni in cui un cifrario a flusso è più appropriato. Un tipico cifrario a flusso crittografa il testo in chiaro un byte alla volta, sebbene un cifrario a flusso possa essere progettato per operare su un bit alla volta o su unità più grandi di un byte alla volta.

In questa struttura, una chiave viene inserita in un generatore di bit pseudocasuali che produce un flusso di numeri a 8 bit che sembrano casuali. Un flusso pseudocasuale è un flusso imprevedibile senza la conoscenza della chiave di input e che ha un carattere apparentemente casuale. L'output del generatore, chiamato **keystream**, viene combinato un byte alla volta con il flusso di testo in chiaro utilizzando l'operazione bit a bit OR esclusivo (XOR). Con un generatore di numeri pseudocasuali progettato correttamente, un cifrario a flusso può essere sicuro quanto un cifrario a blocchi di lunghezza di chiave comparabile. Il vantaggio principale di un cifrario a flusso è che i cifrari a flusso sono quasi sempre più veloci e utilizzano molto meno codice rispetto ai cifrari a blocchi. Il vantaggio di un cifrario a blocchi è che è possibile riutilizzare le chiavi. Per le applicazioni che richiedono la cifratura/decifrazione di un flusso di dati, come su un canale di comunicazione dati o un collegamento browser/Web, un cifrario a flusso potrebbe essere l'alternativa migliore. Per le applicazioni che gestiscono blocchi di dati, come il trasferimento di file, l'e-mail e i database, i cifrari a blocchi potrebbero essere più appropriati. Tuttavia, entrambi i tipi di cifrario possono essere utilizzati praticamente in qualsiasi applicazione.

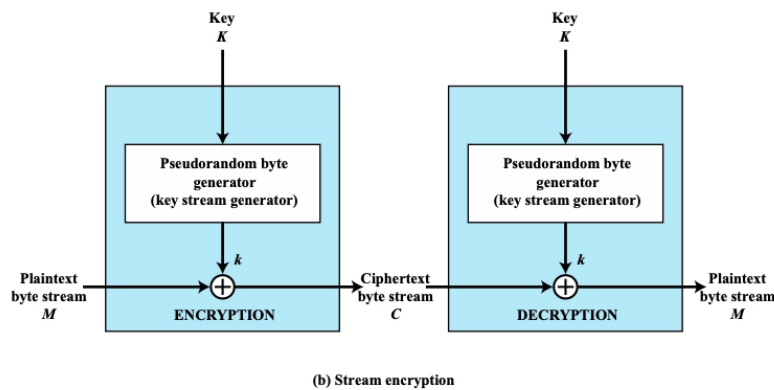


Figura 1: Stream encryption

Autenticazione del messaggio e funzione HASH

L'autenticazione del messaggio può avvenire in due modalità differenti:

Autenticazione del Messaggio con Cifratura

Sembrerebbe possibile eseguire l'autenticazione semplicemente mediante l'uso della crittografia simmetrica. Se assumiamo che solo il mittente e il destinatario condividano una chiave (come dovrebbe essere), allora solo il mittente genuino sarebbe in grado di crittografare con successo un messaggio per l'altro partecipante, a condizione che il destinatario possa riconoscere un messaggio valido. Inoltre, se il messaggio include un codice di rilevamento degli errori e un numero di sequenza, il destinatario ha la certezza che non sono state apportate alterazioni e che la sequenza è corretta. Se il messaggio include anche un timestamp (marca temporale), il destinatario ha la certezza che il messaggio non è stato ritardato oltre quanto normalmente previsto per il transito in rete.

In effetti, la sola crittografia simmetrica non è uno strumento adatto per l'autenticazione dei dati. Per fare un semplice esempio, nella modalità di cifratura ECB, se un attaccante riordina i blocchi di testo cifrato, ogni blocco verrà comunque decifrato con successo. Tuttavia, il riordino può alterare il significato della sequenza di dati complessiva. Sebbene i numeri di sequenza possano essere utilizzati a un certo livello (ad esempio, ogni pacchetto IP), in genere non accade che un numero di sequenza separato sia associato a ciascun blocco di testo in chiaro da b bit. Pertanto, il riordino dei blocchi è una minaccia.

Autenticazione del Messaggio senza Cifratura del Messaggio

In questa sezione, esaminiamo diversi approcci all'autenticazione del messaggio che non si basano sulla cifratura del messaggio. In tutti questi approcci, viene generato un tag di autenticazione e allegato a ciascun messaggio per la trasmissione. Il messaggio stesso non è crittografato e può essere letto a destinazione indipendentemente dalla funzione di autenticazione a destinazione. Poiché gli approcci discussi in questa sezione non crittografano il messaggio, la riservatezza del messaggio non è fornita. Vengono suggerite tre situazioni in cui l'autenticazione del messaggio senza riservatezza è preferibile:

1. Esiste un certo numero di applicazioni in cui lo stesso messaggio viene trasmesso a un numero di destinazioni. Due esempi sono la notifica agli utenti che la rete non è ora disponibile e un segnale di allarme in un centro di controllo. È più economico e affidabile avere una sola destinazione responsabile del monitoraggio dell'autenticità.
2. Un altro possibile scenario è uno scambio in cui una parte ha un carico pesante e non può permettersi il tempo di decifrare tutti i messaggi in arrivo. L'autenticazione viene eseguita su base selettiva, con messaggi scelti a caso per il controllo.

3. L'autenticazione di un programma per computer in chiaro è un servizio interessante. Il programma per computer può essere eseguito senza doverlo decifrare ogni volta, il che comporterebbe uno spreco di risorse del processore. Tuttavia, se al programma fosse allegato un tag di autenticazione del messaggio, esso potrebbe essere controllato ogni volta che è richiesta la garanzia dell'integrità del programma.

Pertanto, c'è posto sia per l'autenticazione che per la cifratura nel soddisfare i requisiti di sicurezza.

CODICE DI AUTENTICAZIONE DEL MESSAGGIO Una tecnica di autenticazione prevede l'uso di una chiave segreta per generare un piccolo blocco di dati, noto come codice di autenticazione del messaggio (MAC), che viene allegato al messaggio. Questa tecnica presuppone che due parti comunicanti, diciamo A e B, condividano una chiave segreta comune K_{AB} . Quando A ha un messaggio da inviare a B, calcola il codice di autenticazione del messaggio come una funzione complessa del messaggio e della chiave:

$$MAC_M = F(K_{AB}, M)^3$$

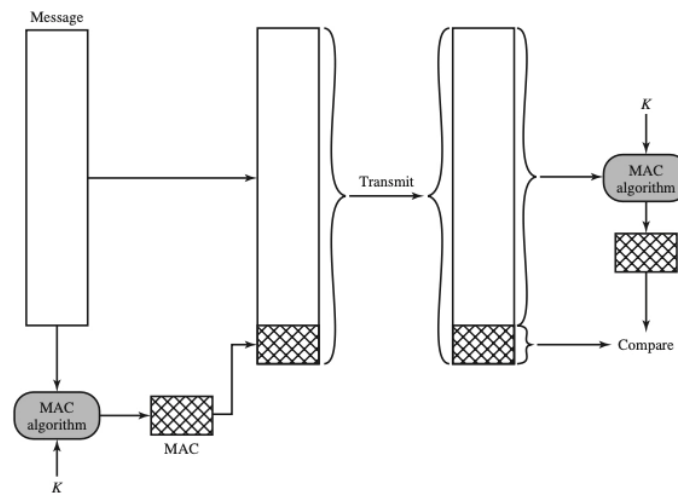


Figura 2: Il messaggio più il codice vengono trasmessi al destinatario previsto.

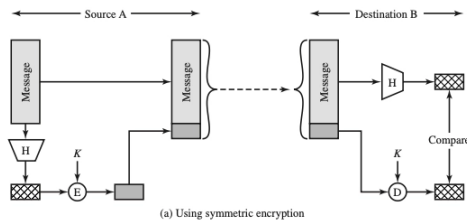
Il messaggio più il codice vengono trasmessi al destinatario previsto. Il destinatario esegue lo stesso calcolo sul messaggio ricevuto, utilizzando la stessa chiave segreta, per generare un nuovo codice di autenticazione del messaggio. Il codice ricevuto viene confrontato con il codice calcolato.

Se assumiamo che solo il destinatario e il mittente conoscano l'identità della chiave segreta e se il codice ricevuto corrisponde al codice calcolato, allora:

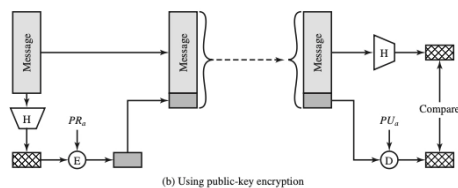
1. Il destinatario è sicuro che il messaggio non è stato alterato. Se un attaccante altera il messaggio ma non altera il codice, il calcolo del codice da parte del destinatario differirà dal codice ricevuto. Poiché si presume che l'attaccante non conosca la chiave segreta, l'attaccante non può alterare il codice per farlo corrispondere alle alterazioni nel messaggio.
2. Il destinatario è sicuro che il messaggio proviene dal presunto mittente. Poiché nessun altro conosce la chiave segreta, nessun altro potrebbe preparare un messaggio con un codice corretto.
3. Se il messaggio include un numero di sequenza, il destinatario può essere rassicurato sulla sequenza corretta, perché un attaccante non può alterare con successo il numero di sequenza.

FUNZIONE HASH UNIDIREZIONALE (ONE-WAY HASH FUNCTION) Un'alternativa al codice di autenticazione del messaggio è la funzione hash unidirezionale. Anche una funzione hash accetta un messaggio di dimensione variabile M come input e produce un digest del messaggio di dimensione fissa $H(M)$ come output. Tipicamente, il messaggio viene completato (padding) fino a un multiplo intero di

una dimensione fissa (ad esempio, blocchi da 1024 bit) e il padding include il valore della lunghezza del messaggio originale in bit. Il campo della lunghezza è una misura di sicurezza per aumentare la difficoltà per un attaccante di produrre un messaggio alternativo con lo stesso valore hash. Ma a differenza del MAC, una funzione hash non prende una chiave segreta come input. Ci sono tre modi per autenticare un messaggio utilizzando la chiave hash:

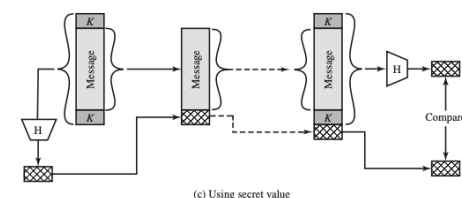


1. Il digest del messaggio può essere crittografato utilizzando la crittografia simmetrica. Se si assume che solo il mittente e il destinatario condividano la chiave di cifratura, allora l'autenticità è assicurata.



2. Il digest del messaggio può anche essere crittografato utilizzando la crittografia a chiave pubblica. Questo implica due vantaggi:

- Firma digitale
- Non distribuzione delle chiavi



3. MAC hash con chiave: presuppone che due parti comunicanti, diciamo A e B, condividano una chiave segreta comune K. Questa chiave segreta è incorporata nel processo di generazione di un codice hash.

In questo terzo modo:

Il mittente calcola: $MD_M = H(K \parallel M \parallel K)$ e invia $[M \parallel MD_M]$

Il ricevitore conosce K, ricalcola $H(K \parallel M \parallel K)$ e verifica MD_M

Requisiti delle funzione HASH

La funzione hash unidirezionale, o funzione hash sicura, è importante non solo nell'autenticazione del messaggio ma anche nelle firme digitali. Lo scopo di una funzione hash è produrre un'impronta digitale ("fingerprint") di un file, messaggio o altro blocco di dati. Per essere utile per l'autenticazione del messaggio, una funzione hash H deve avere le seguenti proprietà:

- H può essere applicata a un blocco di dati di qualsiasi dimensione.
- H produce un output di lunghezza fissa.
- $H(x)$ è relativamente facile da calcolare per ogni dato x, rendendo pratiche le implementazioni sia hardware che software.
- Per ogni dato codice h, è computazionalmente impossibile trovare x tale che $H(x) = h$. Una funzione hash con questa proprietà è definita **unidirezionale (one-way)** o **resistente alla pre-immagine (preimage resistant)**.
- Per ogni dato blocco x, è computazionalmente impossibile trovare $y \neq x$ con $H(y) = H(x)$. Una funzione hash con questa proprietà è definita **resistente alla seconda pre-immagine (second preimage resistant)**. Questa è talvolta indicata come resistenza debole alle collisioni.

- È computazionalmente impossibile trovare una coppia (x,y) tale che $H(x) = H(y)$. Una funzione hash con questa proprietà è definita **resistente alle collisioni (collision resistant)**. Questa è talvolta indicata come resistenza forte alle collisioni.

Tipo di Funzione	Proprietà Soddisfatte	Protezione Offerta
Debole	Dalla 1 ^a alla 5 ^a	Protegge da modifiche a messaggi esistenti.
Forte	Include anche la 6 ^a Proprietà (Resistenza alle collisioni)	Impedisce che qualcuno crei da zero due messaggi diversi con lo stesso hash.

Le funzioni hash non sono tutte uguali; la loro forza dipende da quante di queste proprietà riescono a soddisfare:

- **Proprietà Unidirezionale (4^a Proprietà)** È un "senso unico" matematico. È facile calcolare l'hash di un messaggio, ma è virtualmente impossibile fare il percorso inverso per trovare il messaggio originale partendo dal codice.

- **Perché è vitale:** Se un attaccante potesse invertire la funzione, potrebbe scoprire la chiave segreta K intercettando una trasmissione come

$$MD_M = H(K \parallel M \parallel K)$$

Una volta invertito l'hash in

$$H^{-1}(MD_M)$$

recuperare la chiave K diventerebbe un gioco da ragazzi.

- **Resistenza alla Seconda Pre-immagine (5^a Proprietà)** Garantisce che, dato un messaggio specifico, sia impossibile trovarne un altro differente che produca lo stesso identico valore hash.

- **Perché è vitale:** Impedisce la contraffazione. Senza questa proprietà, un hacker potrebbe intercettare un messaggio e il suo hash, e poi sostituire il messaggio originale con uno fasullo che ha lo stesso "identikit" (hash).

Crittografia Asimmetrica - a chiave pubblica

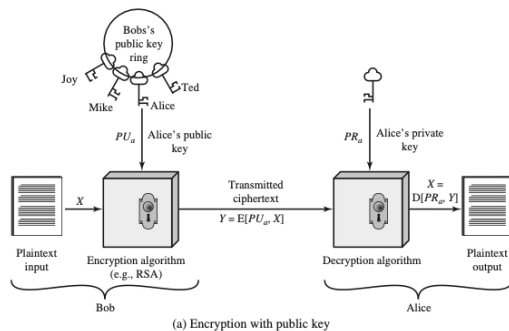
L'applicazione di questa crittografia trova impiego nell'autenticazione dei messaggi e nella distribuzione delle chiavi. Gli algoritmi a chiave pubblica si basano su funzioni matematiche piuttosto che su semplici operazioni su schemi di bit, come quelle utilizzate negli algoritmi di crittografia simmetrica. Cosa più importante, la crittografia a chiave pubblica è **asimmetrica**, comportando l'uso di due chiavi separate, in contrasto con la crittografia simmetrica, che utilizza una sola chiave. L'uso di due chiavi ha profonde conseguenze nelle aree della riservatezza, della distribuzione delle chiavi e dell'autenticazione. Uno schema di crittografia a chiave pubblica ha sei ingredienti:

- **Testo in chiaro (Plaintext):** Questo è il messaggio leggibile o i dati che vengono inseriti nell'algoritmo come input.
- **Algoritmo di cifratura:** L'algoritmo di cifratura esegue varie trasformazioni sul testo in chiaro.
- **Chiave pubblica e privata:** Si tratta di una coppia di chiavi che sono state selezionate in modo che se una viene utilizzata per la cifratura, l'altra viene utilizzata per la decifratura. Le esatte trasformazioni eseguite dall'algoritmo di cifratura dipendono dalla chiave pubblica o privata fornita come input.

- **Testo cifrato (Ciphertext):** Questo è il messaggio rimescolato prodotto come output. Dipende dal testo in chiaro e dalla chiave. Per un dato messaggio, due chiavi diverse produrranno due testi cifrati diversi.
- **Algoritmo di decifratura:** Questo algoritmo accetta il testo cifrato e la chiave corrispondente e produce il testo in chiaro originale.

Come suggeriscono i nomi, la chiave pubblica della coppia viene resa pubblica affinché altri possano usarla, mentre la chiave privata è nota solo al suo proprietario. Un algoritmo crittografico a chiave pubblica per scopi generali si affida a una chiave per la cifratura e a una chiave diversa ma correlata per la decifratura.

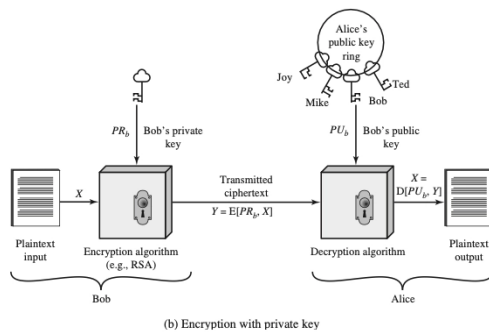
Cifratura per la Riservatezza: Chiave Pubblica



l'obiettivo principale è fare in modo che solo il destinatario possa leggere il contenuto del messaggio. Il mittente prende il messaggio originale e, consultando un elenco di chiavi ottenute da altri utenti, seleziona la **chiave pubblica del destinatario**. Utilizzando un algoritmo di cifratura, il messaggio viene trasformato in un formato rimescolato e illeggibile che viene poi trasmesso.

Una volta ricevuto, il destinatario utilizza la propria **chiave privata** per invertire il processo e riportare il messaggio alla sua forma originale. Poiché solo il destinatario legittimo possiede quella specifica chiave privata, nessun altro può decifrare la comunicazione, garantendo così la segretezza dei dati.

Cifratura per l'Autenticazione e l'Integrità: Chiave Privata



In questo secondo caso, l'obiettivo è dimostrare chi ha inviato il messaggio e assicurarsi che non sia stato manomesso. Il mittente cifra il messaggio originale utilizzando la propria **chiave privata** invece di quella del destinatario. Il messaggio rimescolato viene quindi inviato a destinazione. Il destinatario, per leggere il messaggio, deve utilizzare la **chiave pubblica** del mittente.

Se il destinatario riesce a recuperare con successo il testo originale, ha la certezza matematica che solo il proprietario di quella chiave privata può averlo generato, confermando l'identità del mittente. Inoltre, poiché solo il mittente può creare un codice valido con la propria chiave privata, questo metodo garantisce che nessuno possa modificare il messaggio senza rendere impossibile la sua successiva decifratura con la chiave pubblica.

Algoritmi cifratura asimmetrica

- **RSA (Rivest, Shamir, Adleman), 1977:** L'approccio più ampiamente accettato e implementato. Cifrario a blocchi in cui testo in chiaro e cifrato sono interi tra 0 e n-1.
- **Scambio di chiavi Diffie-Hellman, 1976:** Consente a due utenti di generare un segreto condiviso da usare come chiave per la successiva cifratura simmetrica. Limitato allo scambio di chiavi.
- **Digital Signature Standard (DSS):** Fornisce solo la funzione di firma digitale con SHA-1. Non può essere usato per cifratura o scambio di chiavi.

- **Crittografia a Curve Ellittiche (ECC):** Sicurezza simile a RSA, ma con chiavi molto più piccole.

Algoritmo	Firma Digitale	Distribuzione Chiave Simmetrica	Cifratura Chiavi Segrete
RSA	Sì	Sì	Sì
Diffie-Hellman	No	Sì	No
DSS	Sì	No	No
Curve Ellittiche	Sì	Sì	Sì

Firme Digitali e Gestione Chiavi

Gli algoritmi a chiave pubblica sono utilizzati in una varietà di applicazioni. In termini generali, queste applicazioni rientrano in due categorie: firme digitali e varie tecniche relative alla gestione e distribuzione delle chiavi. Rispetto alla gestione e distribuzione delle chiavi, vi sono almeno tre aspetti distinti nell'uso della crittografia a chiave pubblica a questo riguardo:

- La distribuzione sicura delle chiavi pubbliche.
- L'uso della crittografia a chiave pubblica per distribuire chiavi segrete.
- L'uso della crittografia a chiave pubblica per creare chiavi temporanee per la cifratura dei messaggi.

Firma Digitale

La crittografia a chiave pubblica può essere utilizzata per l'autenticazione tramite la tecnica della **firma digitale**. Secondo lo standard NIST FIPS PUB 186-4, una firma digitale è il risultato di una trasformazione crittografica di dati che fornisce:

- **Autenticazione dell'origine:** verifica chi ha firmato il documento.
- **Integrità dei dati:** garantisce che il messaggio non sia stato alterato.
- **Non ripudio:** il firmatario non può negare di aver apposto la firma.

Algoritmi standard (FIPS 186-4):

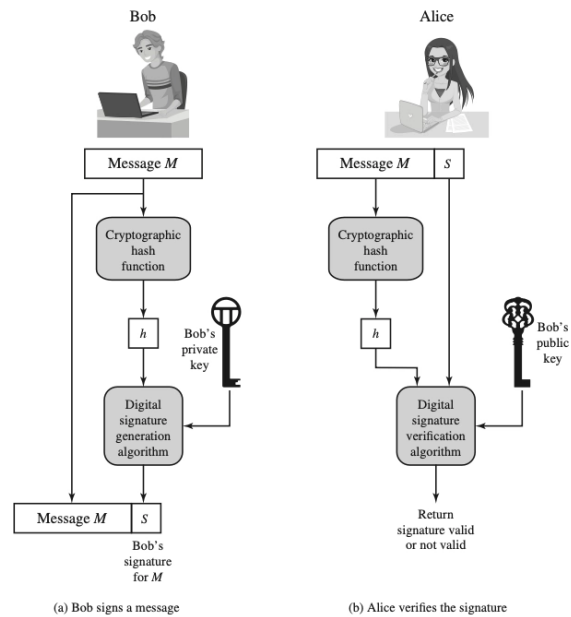
- **DSA (Digital Signature Algorithm):** basato sulla difficoltà del calcolo dei logaritmi discreti.
- **RSA Digital Signature Algorithm:** basato sull'omonimo algoritmo a chiave pubblica.
- **ECDSA (Elliptic Curve Digital Signature Algorithm):** basato sulla crittografia a curve ellittiche.

Il processo:

Ecco come Bob firma un messaggio per Alice assicurandone l'autenticità:

(a) Bob firma il messaggio:

1. **Messaggio (M):** Bob prepara il contenuto.
2. **Hash:** Il messaggio passa attraverso una funzione **hash** crittografica (es. SHA-512) per generare un codice breve (h).
3. **Generazione Firma:** Il codice **hash** (h) e la **chiave privata di Bob** vengono inseriti nell'algoritmo di generazione.
4. **Invio:** Bob invia ad Alice il messaggio originale (M) insieme alla firma digitale (S) appena creata.



(b) Alice verifica la firma:

1. **Ricezione:** Alice riceve il messaggio (M) e la firma (S).
2. **Calcolo Hash:** Alice calcola autonomamente l'**hash** del messaggio ricevuto.
3. **Verifica:** Alice inserisce l'**hash** calcolato, la firma ricevuta e la **chiave pubblica di Bob** nell'algoritmo di verifica.
4. **Risultato:** Se l'algoritmo conferma la validità, Alice ha la certezza che il messaggio è stato firmato da Bob e non è stato manomesso.

Numeri Casuali

I numeri casuali sono essenziali per generare gli strumenti critici della crittografia:

- **Generazione di chiavi:** Vengono usati per creare chiavi per algoritmi a chiave pubblica (come RSA) e chiavi di sessione temporanee.
- **Cifrari a flusso:** Servono a generare lo "stream key" per la cifratura simmetrica a flusso.
- **Protocolli di rete:** Sono utilizzati nell'handshaking (es. Kerberos) per prevenire gli attacchi di *replay*.
- **Buste digitali:** Supportano la creazione di chiavi simmetriche "una tantum".

Per essere sicura, una sequenza di numeri deve soddisfare due criteri spesso difficili da conciliare:

1. **Casualità (Randomness):**

- **Distribuzione uniforme:** Ogni numero della sequenza deve apparire con la stessa frequenza.
- **Indipendenza:** Nessun valore della sequenza può essere dedotto dagli altri (una proprietà difficile da provare matematicamente).

2. **Imprevedibilità (Unpredictability):**

- Un avversario non deve essere in grado di prevedere gli elementi futuri della sequenza basandosi su quelli precedenti. Questo è vitale nelle applicazioni di autenticazione.

Caratteristica	Numeri Pseudocasuali (PRNG)	Veri Numeri Casuali (TRNG)
Origine	Algoritmi deterministici.	Processi fisici non deterministici.
Qualità	Superano i test statistici di casualità.	Casualità "pura" derivata dalla natura.
Prevedibilità	Probabilmente prevedibili se si conosce il seme/algoritmo.	Imprevedibili per definizione.
Esempi	Software e funzioni algoritmiche.	Rumore termico, radiazioni, effetti quantistici, sensori CCD.